

Mid 1

CONCEPTS1) Basic Syntax:-

#include <iostream> // Input/output stream
using namespace std;
int main()

#include "headerfile."

→ means not necessary to write std. in std functions.

```
{
    cout << "Hello world!";
    return 0;
}
```

cin >> variable;

2) Local/global variables:-

↳ A variable within a function etc.

To access global variable we use :: → This variable name
without so it prints variable value.

Post increment → a++

Here a value of print then

increment

++ a

first value increase then print.

3) Reference Variable:-OR TYPE casting:-

float x = 455

so → cout << "The value of a is" << (float) a;

float &y = x;

4) CONSTANT:-

const int a = 3;

5) Same syntax for selection, loop statements, Same syntax for loop

Switch (expression) {

case 1:

{

default {

}

6) pointers:-

* → for dereferencing.

& → for address of operator

7) Structures:-

struct employee

{

}; Then in int main same like struct

OOPs:-

classes, structure.

- To write a function outside class which is declared inside class :-
→ void Employee :: ^{→ scope resolution} setData(int a, int b, int c); (Important in forward declaration)
- for private we use get, set for public we could general use it.

Constructors:-

For constructors with parameters we could take values and initialize or a constructor initialized outside.

Copy constructors:-

"Makes copy of another constructor"

ConstructorName(ConstructorName &obj)

{

a = obj.a

}

"Constructor Name 23 = 2 → copy constructor called because in object in 1 line assigned"

Constructor overloading or as called it executes the constructor as called.

"Destructor → Never takes parameters or not return values."

C++ objects Memory Allocation & using arrays in classes:-

Syntax:- class shop "By default private"

{ int ItemID[100];

int Price[100];

int counter;

}

public class Employee { int Employee::count;

Static Data members & methods:- To save like a counter value. → for all objects in a class

like if we want to count No. of employees we can't pass one object to another.

• Static variable is outside class → Syntax:- in class we have to write in static int count;

↳ By default 0. → "Means that this class ^{me. total objects} will have been counted"

• Static functions when to access only static members:-

```
static void getcount()
{
    cout << "The value of count is" << count << endl;
}
```

• To use this getcount in main → we have to write Employee :: getcount();

ARRAY OF OBJECTS & Passing objects as Parameters

in → main → class name variable[4];

void functionname(classname object)

```
for (
{
    variable[i].function();
}
```

This Pointer:-

this →

/// ↳ This is a pointer which is a pointer which points to the object which invokes the member function

```
class A {
```

```
    int a;
```

```
    public:
```

```
        void setData(int a)
```

```
        {
```

```
            a = a
```

```
        }
```

so alternative is $this \rightarrow a = a$

→ if we do this then this is wrong as it will set garbage value.

or A & void setData(int a)

```
{
    this → a = a;
```

```
    return *this
```

```
}
```

→ It will return an object now

so we can use it in main to access different functions etc

UML class diagram:-

class name
Attributes
Member function

In this for Public → (+) → for functions → (+) ^{parameter name} : void

for Private → (-)

for Protected → (#)

datatype

visibility mode variable name ; (datatype)

Page Victory

Object's Relations:-

friend function $\rightarrow$ Syntax $\rightarrow$ friend classname functionname()
{
3
}

- Not in scope of class, since it is not in the scope of the class it cannot be called from the object of that class."
- Can be invoked without the help of any object.
- Usually contains arguments as objects.

(Relations in C++):- Association $\rightarrow$ No dependency of 2 or more objects on each other. No one is an owner of other.
 $\hookrightarrow$ "using"

Object Pointers:-

class Name * p1 = &c1;

To access it $\rightarrow$ (*p1).membername or c $\rightarrow$ aq = 68.9;

To pass a pointer into function

void change (class Name * c)
{
(*c). aq = 77.2;
}

int * arr = new int(5);

DYNAMIC allocation:-

int * ptr = new int(5*5); $\rightarrow$ dynamic allocation.

classname * c1 = new classname ();

- Constants:- for constant functions they can not access non-constant attributes etc.
- Except for constructor.
- "Constant objects can only constant functions else give error."

Inheritance concepts -> Reusability increases. (if change at one changes in all).

for a derived class

- 1). Multiple inheritance -> more than 1 base class. -> single child, more parent
- 2). Hierarchical -> 1 Parent, 2 child
- 3). Multi level -> Deriving a class from already Derived class.
- 4). Hybrid -> combination of multiple inheritance & multilevel inheritance.
- 5). Single inheritance -> single parent 1 child

"Default visibility mode is Private"

Public/Private.

Syntax - class {derived class - Name} : {visibility mode} {base class - Name}

Visibility mode -> if it is public then all attributes etc of Base class of Public are in Public of Derived class.

if Private then Public members of Base becomes members of Derived class as Private.

Protected -

-> It means that if a class member we want that it should not be allowed to someone to see then we mean it's private but we still want it to be inherited then Protected comes.

B
A
C
D

For Protected members:

Public declaration

Private declaration

Protected

1- Private members

NOT inherited

not inherited

Non inherited

2- Protected members

Protected

Private

Protected

3- Public members

Public

Private

Protected

SYNTAX for multiple inheritance

class Derived : visibility mode

-> Imp (Reuse).

"If we derive a class in private mode then we can access things until we derive that class to another in any mode it cannot access elements or members."

Static Inheritance - (Static functions cannot be overridden).

In terms of static inheritance they are inherited but they are shared across all instances of class.

any changes to static it is reflected in one copy belongs to class, not instance of it.

(through object as well can be accessed)

Page ☐ Victory

CONSTRUCTOR IN Derived class:-

- If more than one value/arguments Derived class then have to show.
- If both have constructors then base class constructor is executed first.
- In multiple inheritance → Base classes are constructed in order as appear
- In multilevel constructor are executed in order of inheritance.

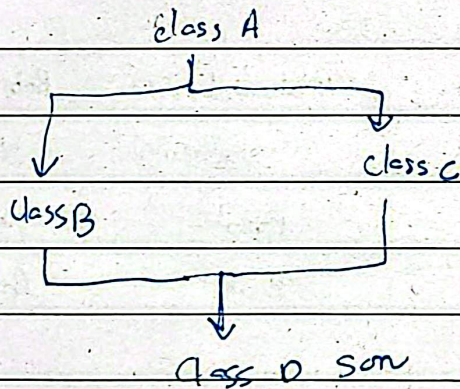
DerivedClassName (args,): BaseClassName constructor (args, ...), Base2-ctor

↳ The constructors are called based on the order if Base1 first then its constructor first then Base2.

How to resolve ambiguity:-

- If a derived function is written that is same as main class so derived class will overwrite the base class function.
- Also you can use `::` To call any function in

Virtual Base class:-



Class B: Virtual
Public A;

}

It is basically done to avoid twice copy of a variable so it's only one time copy is given so D can use.

virtual constructor is given precedence above other constructors.

Initialization list:-

constructor (argument-list) : initialization list

class Test

```
{
  int a;
  int b;
```

public:

→ initialization list.

```
Test (int i, int j) : a(i), b(j) { }
```

or Test (int i, int j) : a(i), b(a+i)

```
{;
```

// Initialize as variables are declared.

Pointer in oop:-

Datatype * variableName = new ^{Datatype} ~~int~~ (40)

→ Normal brackets so if we prints it prints 40.

But if we want to allocate a space then:-

Datatype * variableName = new Datatype[4] → it builds up 4 spaces

To free DMIAB- delete [] arr;

To Normal variable & delete arr;

Pointers uses in classes:-

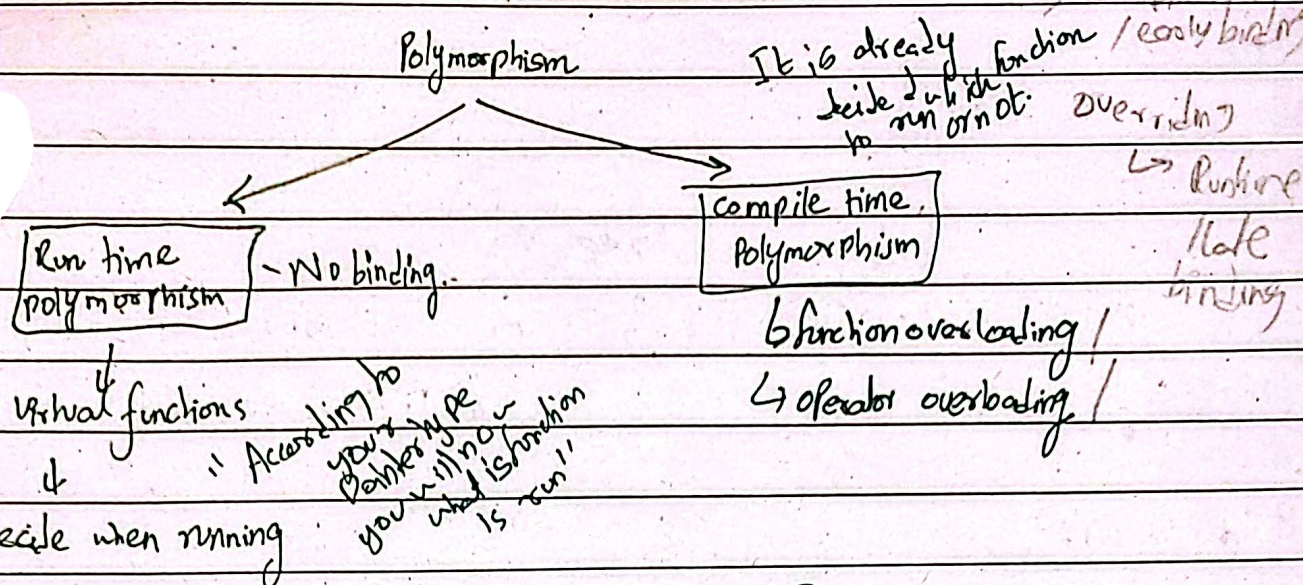
class Name * ptr = new class Name;

Then we can print values using * (*ptr).setData(1, 54); // Ex:

or ptr → setData(1, 54);

Polymorphism:-

"ONE Name multiple forms"



Base class can be pointer point to derived :-

syntax:-

```

Baseclass * base-class-Pointer;
Baseclass obj-base;
Derivedclass obj-derived;
base-class-Pointer = &obj-derived // Pointing base class to derived class
    
```

we did [SS] *

Base class pointer derived to derived class we cannot write both static and virtual together

To change this error :-

use virtual basically in this if base class pointer is pointed to derived class object and we want to print derived class object then we use object-virtual so here we can access derived one.

1) In function overloading definition of it need to be correct

3) Constant functions overloads from constant objects else vice versa

4) Thing to remember = This can't happen that we function that is to be overloaded in one it has in parameter const and other not constant so it will have error

5) If a function is declared as virtual and const both so for overloading it should have const present (Pg #169) needed